数学与 CS 问答笔记

answer-note

今天讲表示不变式（RI）：它在对象生命周期里的义务、一条合格 RI 的五条判据、记录样式（字段旁注释、逐条布尔条件、与 checkRep 一一对应），以及 `mem` 这类 map 表示的 RI 注释模板；只讲写法与判据，不填项目答案。

October 25, 2026

Contents

1. RI 的写法与记录样式（Rep Invariant: How to Write and Record It） 3
   1.1. 费曼直觉 3
   1.2. 硬核机制 3
      1.2.1. 一、RI 的定义与它和 AF 的关系 3
      1.2.2. 二、RI 在对象生命周期里的义务 3
      1.2.3. 三、一条合格 RI 的五条判据 3
      1.2.4. 四、RI 的记录样式 4
      1.2.5. 五、RI 与 checkRep 的对应 5
      1.2.6. 六、RI 通常覆盖的三类子句 5
      1.2.7. 七、当前例子：`mem` 的 RI 记录样式 6
      1.2.8. 八、常见错误 6
   1.3. 关键点 6
   1.4. 变式检验 6
   1.5. 同类归类 7
   1.6. Go 语言机制补充 7
   1.7. 本次操作与当前状态 7
   1.8. 补充：怎么表达“所有 key 的长度 > 0” 7
      1.8.1. 一、RI 注释里的写法 8
      1.8.2. 二、Go 里的检查模式 8
      1.8.3. 三、这条 RI 合不合格 8
      1.8.4. 四、如果加进 `mem` 的 RI，会连带影响什么 9
      1.8.5. 五、常见写法错误 9

Index of Listings 10

1. RI 的写法与记录样式（Rep Invariant: How to Write and Record It）

1.1. 费曼直觉

把 ADT 想成一栋楼：AF 是“每个房间怎么读”的说明牌，RI 是“哪些房间可以住人”的验收标准。验收标准不能写“房子没问题”，而要写成一条条能打勾/打叉的检查项：承重墙在不在、消防通道通不通、层高够不够。

RI 就是 rep 的验收清单。它只谈内部数据（rep），不谈客户端看到的值（抽象值）；每条都要能判真假；每条都对应代码里的一次检查。

1.2. 硬核机制

1.2.1. 一、RI 的定义与它和 AF 的关系

表示不变式（rep invariant, RI）是 rep 值上的谓词：

```
RI : R -> bool
```

Listing 1: 输入一个 rep 值，输出真或假；为真表示这个内部状态是合法的

它与 AF 的关系只有一句：

> 📌 **核心要点 RI(r) 为真，当且仅当 AF(r) 有定义。** AF 是偏函数，不是所有 rep 值都有抽象值；RI 精确刻画了 AF 的定义域。

课程里的 CharSet 例子：rep 是字符串，抽象值是字符集合。若规定“字符串里不能有重复字符”，那么 `"abbc"` 不满足 RI，也就没有抽象值；`"abc"` 满足 RI，AF 把它解释成集合 $\{a, b, c\}$。

1.2.2. 二、RI 在对象生命周期里的义务

不变式要“活过整个对象生命”。要证明它始终成立，只需三条：

1. **建立（establish）**：创建者（creator）与生产者（producer）在返回前让 RI 为真；
2. **保持（preserve）**：变值器（mutator）、观察者（observer）与生产者在返回前不让 RI 变假；
3. **无表示暴露**：客户端不能绕过方法改 rep。

三条合起来就是结构归纳：创建时真、每一步操作后仍真、没有旁路，那么任意时刻都真。

1.2.3. 三、一条合格 RI 的五条判据

判据	要问什么	反例	合格的样子
只谈 rep	它是否只引用字段与字段的值？	`mem 表示的表没有重复键`、`值都合法`	只出现字段、长度、元素、nil、大小关系

可判真假	代入任意 rep 值，能得到 true 或 false 吗？	`尽量合法`、`结构正确`	每条都是一个布尔表达式
精确	它有没有说清“具体什么条件”?	`字段没问题`	写出字段之间的关系或范围
不过强	它会不会排除规格允许的抽象值？	禁止规格本来允许的空键/空值	只排除“无法解释”的 rep 值
不过弱	它能不能挡住所有非法 rep?	不检查 nil map	每条非法状态都有对应子句

⚠️ **陷阱预警** “过强”与“过弱”是一对相反的坑。过强会让某些合法抽象值造不出来（AF 不满射）；过弱会让非法 rep 溜进来，AF 在它上面没有定义，后续方法只能到处防御。RI 的每一条都要能回答：“它挡住了什么？如果去掉它，哪个合法抽象值会受影响？”

1.2.4. 四、RI 的记录样式

位置：紧挨着字段声明，用普通注释；不要写成 `go doc` 可见的文档注释——那会把内部表示变成公开承诺。

标题与结构：`// Rep invariant:` 之后逐条列条件。每条一行，条件之间是逻辑“与”。可以编号或用短横线，但同一文件内保持一致。

与代码的对应：每条子句对应 `checkRep` 里的一个 `if`。注释里写了却没检查，或者检查了却没写，都是不一致。

```
type Counter struct{ n int }

// Rep invariant:
//   n >= 0
```

Listing 2: 最小样式：字段旁、一行一条、布尔条件；`Counter` 是通用示例，与项目无关

```
type RatNum struct {
  num, den int
}

// Rep invariant:
//   den > 0
//   gcd(|num|, den) = 1
```

Listing 3: 多条子句的样式：每行一个独立的布尔条件，合起来是逻辑与；`RatNum` 来自课程材料

```
type CharSet struct{ s string }

// Rep invariant:
//   s contains no repeated characters
```

Listing 4: 用自然语言写条件也可以，只要能判真假、只谈 rep；`CharSet` 来自课程材料

写法建议：

1. 每条尽量短，一条只说一件事；
2. 条件里出现的名字必须是 rep 字段（或字段的局部量），不能出现“抽象值”“表”“集合”这类词；
3. 如果某条依赖另一个字段，写清两个字段的关系；
4. 空 rep 值（零值、nil）是否合法，要单独有一条或明确排除；
5. 不要写恒真条件（例如 `len(m) >= 0`），它挡不住任何东西。

1.2.5. 五、RI 与 checkRep 的对应

```
func (c *Counter) checkRep() {
  if c.n < 0 {
    panic("demo: Counter.n is negative")
  }
}
```

Listing 5: 每条 RI 子句一个 `if`；Go 没有 `assert`，生产代码用 `panic` 表示“程序坏了”

调用时机：

1. 创建者、生产者、变值器在返回前调用；
2. 观察者也建议调用——如果客户端通过表示暴露改坏了 rep，下一次观察能当场抓住；
3. 测试里可以在每次操作后调用，形成“随机操作 + 检查”的回归网。

1.2.6. 六、RI 通常覆盖的三类子句

类型	问什么	通用例子
结构完整性	字段是否非 nil、长度/排序/唯一性是否成立？	`s` 非 nil；`items` 有序；键不重复
值域约束	字段的取值允许范围是什么？	`den > 0`；`count >= 0`；字符串符合某格式
跨字段一致性	多个字段之间有什么关系？	`len(keys) == len(vals)`；`num/den` 已约分

> 📌 **核心要点** 三类子句不是模板填空，而是提醒你“从哪些角度想”。一条 RI 可以只有一条子句，也可以有五六条；关键是每条都能被建立、被保持、被检查。

1.2.7. 七、当前例子：`mem` 的 RI 记录样式

`mem` 是 map，RI 的**样式**仍然是“字段旁 + 逐条布尔条件”，不会因为它是 map 就改变。可以先把结构写出来，内容留空：

```
// Rep invariant:
//   - <condition on mem itself>
//   - <allowed range of keys>
//   - <allowed range of values>
//   - <lifecycle / cross-field condition>
```

Listing 6: `KV.mem` 的 RI 注释骨架：四行占位，每行一条布尔条件；填什么由你决定

四个占位分别对应上一节的三类子句加上生命周期：

1. `mem` 本身：它的零值/初始化状态算不算合法？
2. key 的允许范围：空键、nil 键（在 Go 里会变成同一个字符串）怎么处理？
3. value 的允许范围：nil 值、空值算不算合法？
4. 生命周期/跨字段：`Open` 之前与之后，合法的 rep 状态是否不同？

> ⚠️ **陷阱预警** 这四行是**结构**，不是答案。每一条的取舍都会反过来影响 `Set` / `Get` / `Del` 的规格与测试；不要为了“填满四行”而硬写。写不出某一条时，先问：“如果去掉它，会出现哪个不该存在的 rep 状态？”

1.2.8. 八、常见错误

1. **把 RI 写成 AF**：`mem 表示一张键值表` ——这是抽象值的描述，不是 rep 的谓词；
2. **引用抽象值**：`值都是合法的` ——“合法”是抽象层的词，RI 只能用 rep 值判断；
3. **写恒真条件**：`len(mem) >= 0` 永远为真，等于没写；
4. **把安全性论证混进来**：`值不与外部共享` ——这是所有权约定，不是 rep 本身的合法性；
5. **RI 与 checkRep 不一致**：注释写了三条，代码只检查一条；
6. **过强**：把规格允许的输入排除掉，导致某些抽象值造不出来；
7. **过弱**：nil map 或 nil 值能进入，AF 在它们上面没有定义。

1.3. 关键点

一句话：**RI 是 rep 上的布尔谓词清单——只谈字段、逐条可判、能被创建者建立、被每个操作保持、被 `checkRep` 检查；它挡住的是“没有抽象值”的 rep 状态，不是“客户端不该传”的参数。**

1.4. 变式检验

1. 一条恒真的条件（如 `len(m) >= 0`）算不算 RI 子句？它对 AF 的定义域有影响吗？
2. 如果 RI 过强，AF 的哪个性质会受影响：满射、单射，还是偏函数性？
3. 为什么 RI 不能引用抽象值？如果引用了，判断一个非法 rep 值时会发生什么？
4. “值不与调用方共享”应该写进 RI 还是安全性论证？两者的判断对象有什么不同？
5. 同一个 rep 类型能否配两组不同的 RI？如果能，说明“类型决定 RI”这句话错在哪？

1.5. 同类归类

概念	回答的问题	典型位置
AF	rep 值表示哪个抽象值？	字段旁注释
RI	哪些 rep 值合法？	字段旁注释 + `checkRep`
安全性论证	客户端能不能拿到并修改 rep?	字段旁注释 + 参数/返回值检查
规格	客户端能观察到什么行为？	导出文档注释 + 测试

1.6. Go 语言机制补充

1. map 的零值是 nil；读 nil map 返回零值，写入 panic（`/usr/lib/go/src/runtime/map.go:21`）。所以“nil 是否合法”往往是一条 RI 子句。
2. `delete` 对 nil map 与缺失键都是 no-op，且没有返回值（`go doc builtin.delete`）。
3. Go 没有 `assert`；RI 检查用 `if !cond { panic(...) }`，测试用 `t.Fatalf`。
4. `panic` 用于“程序自己坏了”（RI 被破坏），可预期的失败用 `error`。
5. 同包测试可以直接调用 `checkRep`，也可以直接改 `mem`——后者会绕过 RI，属于白盒滥用。
6. `go doc ./internal/db` 只显示导出文档；字段旁注释不会出现在里面，这正是我们想要的效果。

1.7. 本次操作与当前状态

1. **新建** `note/2026-10-25.typ`（answer-note 完整头部），写入本主题。
2. **讲解内容**：RI 的定义与 AF 的关系、生命周期三条义务、五条判据、记录样式（字段旁、逐条布尔条件、与 `checkRep` 一一对应）、三类子句、`mem` 的四行占位骨架、常见错误。
3. **未做**：没有填 `KV.mem` 的 RI 内容，没有给练习答案；`internal/db/kv.go` 与 `kv_test.go` 未改动。
4. **验证**：`typst compile` 零错误；`go vet ./...` 通过。
5. **下一步**：你按四行骨架写出自己的 RI，或直接写进代码后交作业；我按契约只做诊断。

1.8. 补充：怎么表达“所有 key 的长度 > 0”

先澄清一个歧义：**“长度 > 0” 与 “是正数” 是同一件事**——长度本身是非负整数，`len(k) > 0` 就是“正数”。如果你说的“正数”是指 **key 本身**（例如 key 是 `int`），那是另一条子句，和长度无关：

你想说的	数学写法	Go 里的判断	适用 KEY 类型
所有 key 非空	`forall k in dom(m), len(k) > 0`	`len(k) == 0` 即违规	string、自定义 string 类型
所有数值 key 为正	`forall k in dom(m), k > 0`	`k <= 0` 即违规	int 等数值类型

所有 key 至少一个字符	`forall k in dom(m), RuneCount(k) > 0`	`utf8.RuneCountInString(k) == 0` 即违规	string

对“非空”判断来说，`len(k) > 0` 与“至少一个字符”等价——空串的字节数和字符数都是 0；差别只在“长度”的语义（字节 vs 字符），不影响这条 RI。

1.8.1. 一、RI 注释里的写法

```
// Rep invariant:
//   every key is non-empty: for all k in dom(m), len(k) > 0
```

Listing 7: 自然语言 + 形式化条件并列；关键是“只谈 rep 的键集合与长度”，不出现抽象值

也可以只写形式化版本：

```
// Rep invariant:
//   forall k in dom(m), len(k) > 0
```

Listing 8: 形式化写法；项目里如果注释统一用英文，就保持英文

1.8.2. 二、Go 里的检查模式

用通用 `map[string]int` 演示，套到你的字段时只需换 map 名与 panic 文案：

```
func checkKeyNonEmpty(m map[string]int) {
  for k := range m {
    if len(k) == 0 {
      panic("demo: empty key in map")
    }
  }
}
```

Listing 9: “所有 key 满足 P” 的通用模式：遍历键集合，对每个键断言 P；一条 RI 子句对应一个循环加一个 if

三条语言事实：

1. `len(string)` 返回**字节数**；非空判断只关心是否为 0，所以字节/字符之分不影响结果；
2. map 的键必须可比较；`[]byte` 不能直接做键，通常先转 `string`；而 `string(nil)` 与 `string([]byte{})` 都是 `""`，在键空间里无法区分；
3. 如果键是自定义类型（`type Key string`），`len(k)` 仍可用；如果是 `type Key int`，就只能写 `k > 0`，不能用 `len`。

1.8.3. 三、这条 RI 合不合格

按五条判据过一遍：

1. **只谈 rep**：合格——只出现键集合与长度；
2. **可判真假**：合格——每个键都能算出布尔值；
3. **精确**：合格——条件明确是“非空”；
4. **不过强**：**取决于你的规格**——如果抽象值空间允许空键，这条就会排除合法输入；

5. **不过弱**：它只挡住空键，不挡其他非法状态；RI 通常需要多条子句合起来。

> 📌 **核心要点** “所有 key 非空”是一条**可选的规格决策**，不是语言要求。加它之前先回答：空键在抽象值里有没有意义？如果允许空键，这条 RI 就是过强的；如果不允许，还要定义 `Set` / `Get` / `Del` 收到空键时的行为（拒绝并返回错误，还是视为非法调用）。

1.8.4. 四、如果加进 `mem` 的 RI，会连带影响什么

1. **key 的产生方式**：`string(key)` 会把 `nil` 与空字节切片都变成 `""`，所以“空键”实际上是一个无法再细分的状态；
2. `Set` **的规格**：空键被拒绝时，`Set` 返回什么？`(false, error)`？还是 panic？这要写进规格；
3. `Get` / `Del` **的规格**：空键查询是“未命中”还是“非法输入”？两者对应不同的返回值形状；
4. **测试分区**：空键子域从“合法但边界”变成“非法输入”，测试策略也要跟着改；
5. `checkRep`：这条子句要有一个对应的 `if`，并在创建者/变值器返回前调用。

> ⚠️ **陷阱预警** 不要为了“让 RI 看起来完整”而加这条。RI 的每条子句都要能回答：“去掉它会出现哪个非法 rep 状态？”如果答案是“空键也能正常表示一张表”，那它就不该在 RI 里。

1.8.5. 五、常见写法错误

1. `len(k) >= 0` ——恒真，挡不住任何东西；
2. `k > 0` 用在 string 上——编译不过，字符串没有大小顺序意义上的正负；
3. `len(k) > 0 && k > 0` ——对 string 后半句无意义；对 int 前半句无意义；
4. 把“key 非空”写成“key 的字符数大于 0”却用 `len` 判断非空——对非空等价，但描述长度时要分清字节与字符；
5. 只在注释里写、`checkRep` 里不检查——注释与代码不一致。

Index of Listings

Listing 1 输入一个 rep 值，输出真或假；为真表示这个内部状态是合法的 3
Listing 2 最小样式：字段旁、一行一条、布尔条件；`Counter` 是通用示例，与项目无关 4
Listing 3 多条子句的样式：每行一个独立的布尔条件，合起来是逻辑与；`RatNum` 来自课程材料 4
Listing 4 用自然语言写条件也可以，只要能判真假、只谈 rep；`CharSet` 来自课程材料 5
Listing 5 每条 RI 子句一个 `if`；Go 没有 `assert`，生产代码用 `panic` 表示“程序坏了” 5
Listing 6 `KV.mem` 的 RI 注释骨架：四行占位，每行一条布尔条件；填什么由你决定 . . . 6
Listing 7 自然语言 + 形式化条件并列；关键是“只谈 rep 的键集合与长度”，不出现抽象值 8
Listing 8 形式化写法；项目里如果注释统一用英文，就保持英文 8
Listing 9 “所有 key 满足 P” 的通用模式：遍历键集合，对每个键断言 P；一条 RI 子句对应一个循环加一个 if 8